

Dockerized Distributed Web Crawler

Polite Crawling, Shared Storage, Streaming Indexing, and Ranked Search

Vaibhav Gupta Sayam Agarwal Aaditya

Project Evaluation Presentation

May 7, 2026

What We Built

Project goal

Build a distributed web crawler that can run from Docker, crawl pages through multiple worker containers, store crawled data safely, update the search index continuously, and return ranked search results through an API.

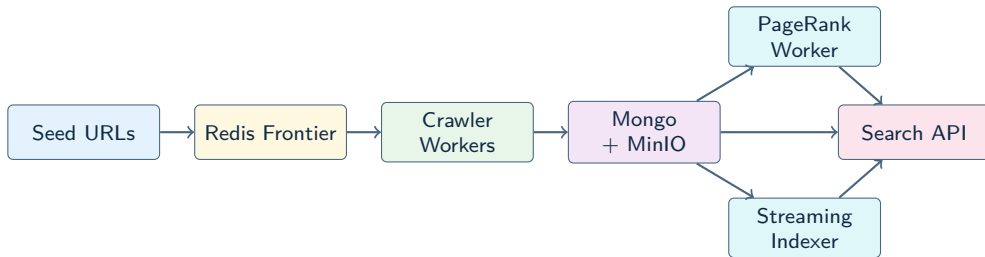
Crawler side

- Redis manages URL scheduling and worker coordination.
- Crawler workers fetch pages independently.
- Robots.txt and per-domain politeness protect target sites.
- MinIO stores raw HTML outside MongoDB.

Search side

- MongoDB stores metadata, clean text, graph state, and indexes.
- Streaming workers update search data after crawls.
- PageRank maintains page authority scores.
- FastAPI combines relevance and authority for results.

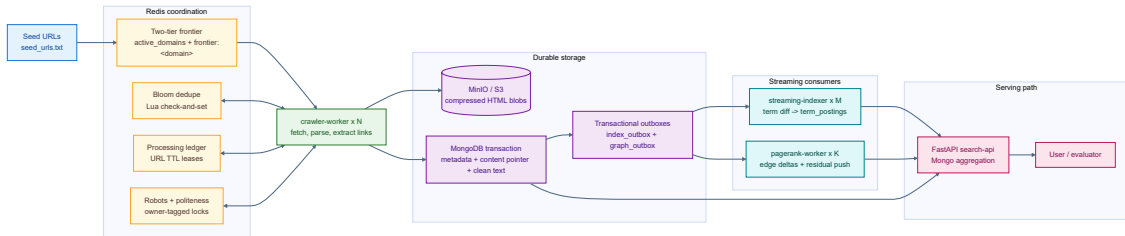
End-to-End Flow



Flow summary

Seed URLs enter Redis. Workers fetch and parse pages. MongoDB commits page data and outbox events atomically. Streaming consumers update search and authority state. FastAPI serves ranked results.

System Architecture



Full architecture: Redis coordination, crawler workers, MinIO blob storage, MongoDB records, streaming consumers, and search API.

Docker Compose Topology

Docker Compose

What it does: Runs Redis, MongoDB, MinIO, crawler workers, streaming indexer, PageRank worker, search API, and seed helper as containers.

Why we used it: It gives a reproducible setup where every service runs in an isolated container with the same network and environment.

What Docker gives us

- Same setup for every evaluator machine.
- Easy service startup and shutdown.
- Multiple crawler workers can run side by side.
- Services communicate using Compose DNS names.

Seed Helper

Seed helper

What it does: Reads `seed_urls.txt` and inserts starting URLs into the Redis frontier.

Why we used it: The crawler is pull-based. Workers need an initial frontier before they can start discovering more links.

Role in the flow

The seed helper only starts the crawl. After that, crawler workers discover new links from downloaded pages and add those URLs back into Redis.

Redis: Coordination Layer

Redis

What it does: Stores the URL frontier, Bloom-filter dedupe bits, URL processing leases, domain locks, and robots/crawl-delay cache.

Why we used it: Redis gives low-latency atomic coordination between many worker containers.

Key Redis structures

- `crawler:active_domains`
- `crawler:frontier:<domain>`
- `crawler:bloom`
- `crawler:processing`
- `lock:<domain>`

Crawler Worker

crawler-worker

What it does: Pulls URLs from Redis, checks politeness, downloads HTML, extracts clean text and outbound links, and writes page data to storage.

Why we used it: It is autonomous and stateless except for shared Redis/Mongo/MinIO state, so many workers can run at once.

Worker loop

- 1 Dequeue URL from Redis.
- 2 Acquire domain politeness lock.
- 3 Fetch page.
- 4 Parse title, text, links.
- 5 Save transaction + outbox events.
- 6 Release URL lease.

Robots.txt And Politeness

Robots and politeness

What it does: Fetches `/robots.txt`, checks `can_fetch`, caches robots content and crawl-delay in Redis, and enforces per-domain locks.

Why we used it: Distributed crawlers must be polite. Multiple workers should not attack the same domain simultaneously.

Policy

If a URL is disallowed, it is not added to the frontier. If robots data is missing or unavailable, the current crawler policy allows crawling by default.

MinIO

What it does: Stores compressed raw HTML blobs in the shared `crawler-html` bucket. MongoDB stores only pointers and metadata.

Why we used it: Large HTML blobs should not bloat MongoDB's working set. Shared object storage also works when workers and API run on different machines.

Data split

MongoDB keeps searchable metadata and references. MinIO keeps the heavier original page payloads.

MongoDB Storage

MongoDB

What it does: Stores page metadata, clean text, content pointers, indexing state, graph edges, and PageRank scores.

Why we used it: MongoDB gives durable structured storage and supports atomic updates across the crawler pipeline.

Main collections

- `pages_metadata`
- `pages_content`
- `pages_documents`
- `index_outbox`
- `graph_outbox`
- `term_postings`, `graph_edges`, `pagerank_nodes`

Streaming Indexer

streaming-indexer

What it does: Consumes `index_outbox`, loads clean text from `pages_documents`, tokenizes/stems terms, computes term frequencies, and updates `term_postings`.

Why we used it: Indexing is moved out of crawler workers, so crawling stays fast and search state is updated asynchronously.

Search index shape

`term_postings` stores one document per `(term, url)`, avoiding giant embedded posting arrays for frequent terms.

Incremental PageRank Worker

pagerank-worker

What it does: Consumes `graph_outbox`, compares new outbound links with historical `graph_edges`, applies PageRank contribution deltas, and updates `pagerank_nodes`.

Why we used it: Authority scoring is expensive graph math; separating it from crawling keeps the crawler responsive.

How authority changes

When a page links to another page, it passes part of its authority forward. If the link graph changes after a recrawl, only the affected contributions are updated.

Search API

FastAPI search-api

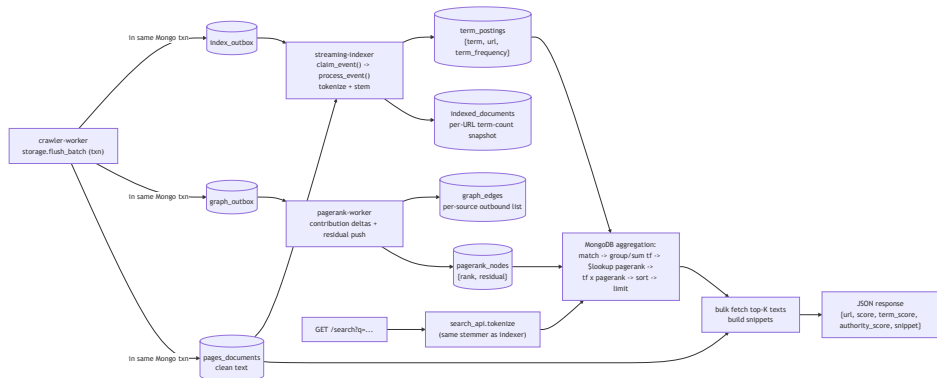
What it does: Receives a query, tokenizes it, aggregates matching `term_postings`, joins `pagerank_nodes`, computes final score, sorts and limits inside MongoDB.

Why we used it: Search stays fast because it does not crawl, tokenize all documents, or recompute PageRank at query time.

Ranking formula

$$\text{final score} = \text{term score} \times \text{authority score}$$

Index And Search Flow



Result

Crawled text becomes searchable postings, PageRank becomes authority, and the API combines both signals when answering a query.

Fault Tolerance

Failures handled

- Worker dies mid-crawl.
- Duplicate URL discovery.
- Domain lock contention.
- Outbox consumer crash.
- PageRank transient write conflict.

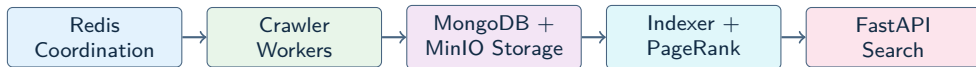
Mechanisms

- Redis TTL leases.
- Lua atomic frontier updates.
- MongoDB transactions.
- Outbox claim/reclaim.
- PageRank recompute reset.

How One Page Moves Through The System

- ➊ A URL is selected from Redis according to priority and domain availability.
- ➋ A crawler worker checks robots.txt and waits for the domain lock if needed.
- ➌ The worker downloads HTML, extracts text, and discovers outbound links.
- ➍ Raw HTML is compressed into MinIO and page metadata is saved in MongoDB.
- ➎ Background workers update the search index and PageRank information.
- ➏ The Search API uses the prepared data to return ranked results quickly.

Final System Summary



Main contribution

The project connects distributed crawling, polite URL scheduling, durable storage, streaming indexing, graph-based authority scoring, and ranked search into one Dockerized pipeline.